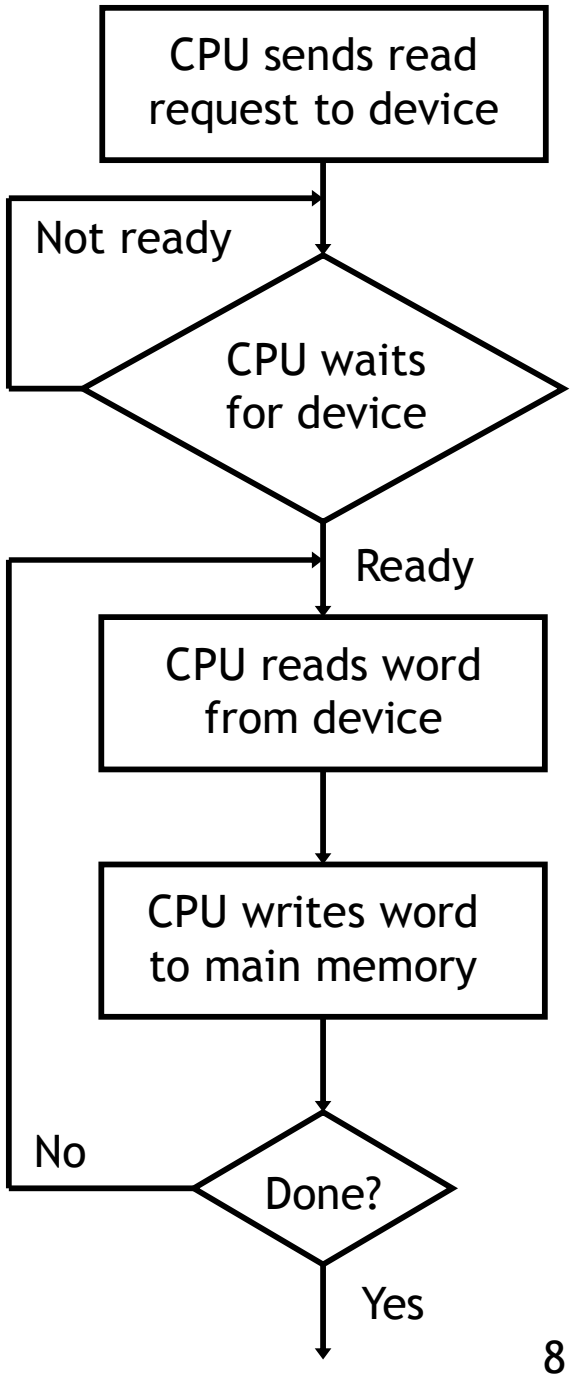
Transferring data with programmed I/O

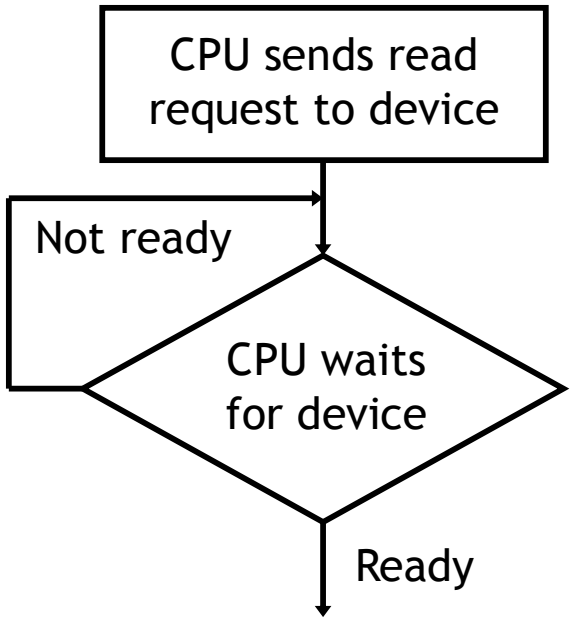
- The second important question is how data is transferred between a device and memory.
- Under programmed I/O, it's all up to a user program or the operating system.
  - The CPU makes a request and then waits for the device to become ready (*e.g.*, to move the disk head).
  - Buses are only 32-64 bits wide, so the last few steps are repeated for large transfers.
- A lot of CPU time is needed for this!
  - If the device is slow the CPU might have to wait a long time—as we will see, most devices *are* slow compared to modern CPUs.
  - The CPU is also involved as a middleman for the actual data transfer.

(This CPU flowchart is based on one from *Computer Organization and Architecture* by William Stallings.)



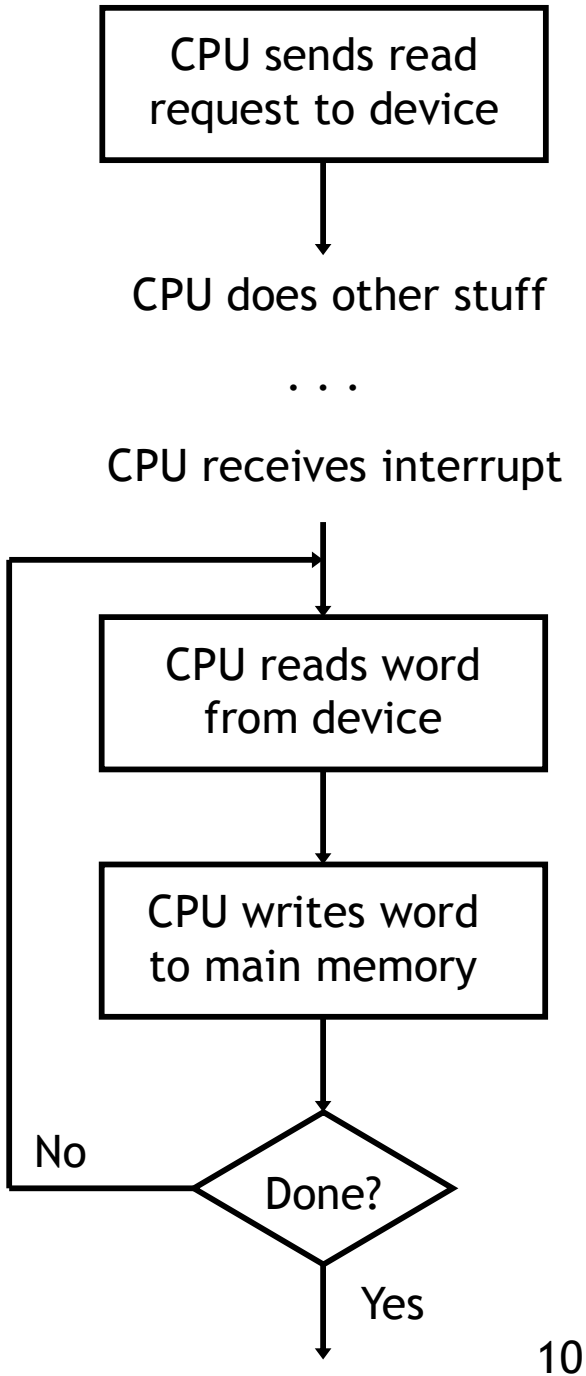
Can you hear me now? Can you hear me now?

- Continually checking to see if a device is ready is called polling.
- It's not a particularly efficient use of the CPU.
  - The CPU repeatedly asks the device if it's ready or not.
  - The processor has to ask often enough to ensure that it doesn't miss anything, which means it can't do much else while waiting.
- An analogy is waiting for your car to be fixed.
  - You could call the mechanic every minute, but that takes up all your time.
  - A better idea is to wait for the mechanic to call *you*.



Interrupt-driven I/O

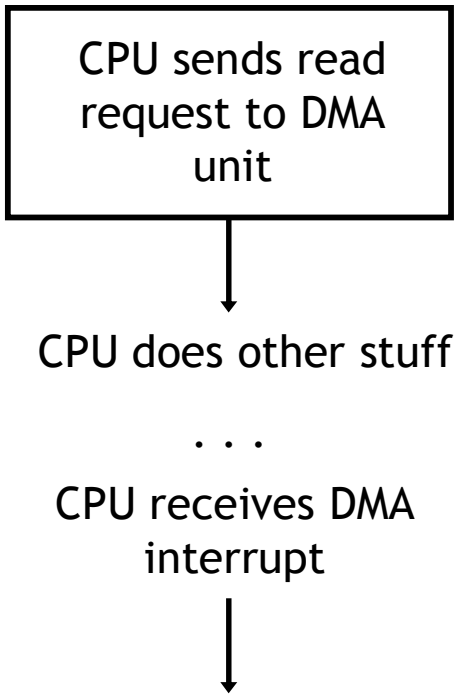
- Interrupt-driven I/O attacks the problem of the processor having to wait for a slow device.
- Instead of waiting, the CPU continues with other calculations. The device interrupts the processor when the data is ready.
- The data transfer steps are still the same as with programmed I/O, and still occupy the CPU.



(Flowchart based on Stallings again.)

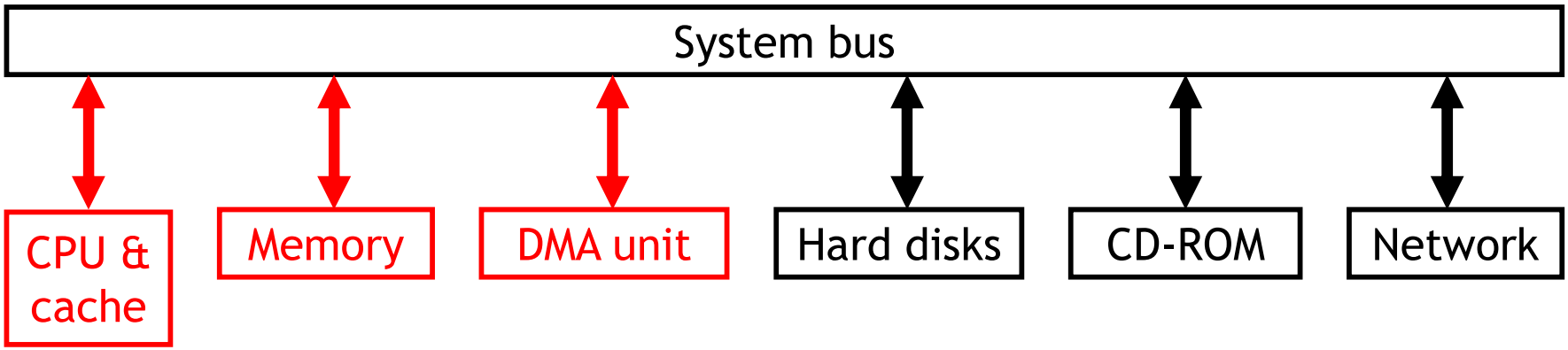
Direct memory access

- One final method of data transfer is to introduce a direct memory access, or DMA, controller.
- The DMA controller is a simple processor which does most of the functions that the CPU would otherwise have to handle.
  - The CPU asks the DMA controller to transfer data between a device and main memory. After that, the CPU can continue with other tasks.
  - The DMA controller issues requests to the right I/O device, waits, and manages the transfers between the device and main memory.
  - Once finished, the DMA controller interrupts the CPU.



(Flowchart again.)

Main memory problems



- As you might guess, there are some complications with DMA.
  - Since both the processor and the DMA controller may need to access main memory, some form of arbitration is required.
  - If the DMA unit writes to a memory location that is also contained in the cache, the cache and memory could become inconsistent.

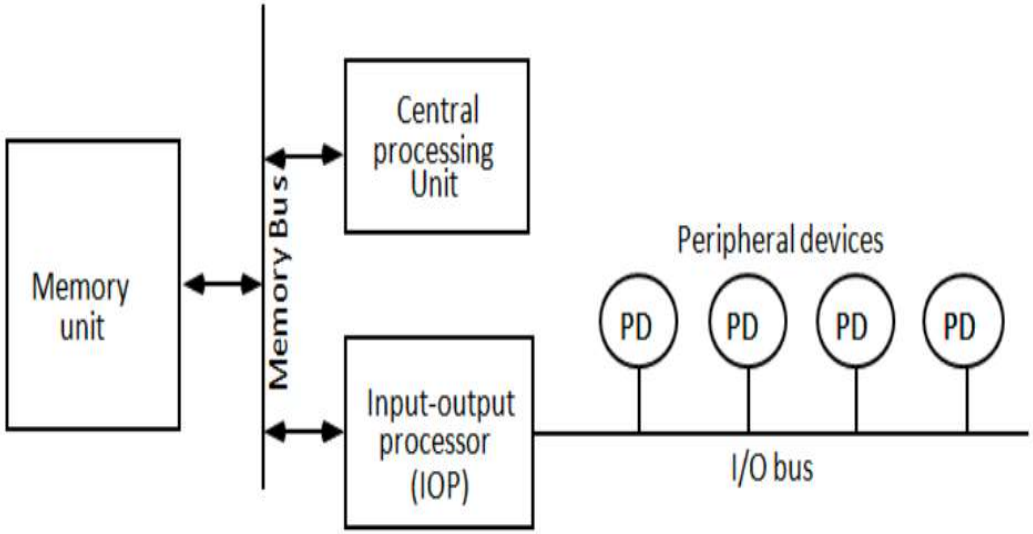
I/O processors

IOP vs CPU ??

- IOP **is similar to a CPU** except that it is **designed to handle the details of I/O processing**.

IOP vs DMA ??

- Unlike the DMA controller that must be setup entirely by the CPU, the IOP can fetch and execute its own instruction.



IOP ....

- The block diagram of a computer

with two processors is shown in figure.

- The memory unit occupies central position an

means of direct memory access.

- The IOP provides a path of for transfer of data

memory unit.

- The CPU is usually assigned

the task of initiating the I/O program.

but I/O instructions are executed in the IOP.